Guía Definitiva de Comandos CSS

Referencia técnica de sintaxis, casos de uso, restricciones y exclusiones mutuas.

1. Modelo de Caja, Posicionamiento y Display

Comando / Propiedad	Sintaxis y Uso	Cuándo Sí Usar	Cuándo NO Usar
`display: flex;`	`display: flex;` Activa el contenedor flexible unidimensional.	Diseño de barras de navegación, alineación de elementos en fila/columna.	Grillas bidimensionales complejas con filas y columnas estrictas (usar Grid).
`position: absolute;`	`position: absolute; top: 10px; left: 20px;` Saca al elemento del flujo normal.	Tooltips, modales flotantes, insignias (badges) sobre elementos.	Diseñar la estructura principal de la página (rompe el flujo responsivo).
`float: left / right;`	`float: left;` Hace que el elemento flote a un lado.	Textos que envuelven imágenes en artículos de blog tradicionales.	Para maquetación web moderna (reemplazado por Flexbox y Grid).
`box-sizing: border-box;`	`box-sizing: border-box;` Incluye padding y bordes en el ancho total.	Requerido globalmente al inicio de cualquier hoja de estilos profesional.	Nunca (su ausencia genera errores de cálculo de ancho con paddings).
`overflow: hidden;`	`overflow: hidden; overflow-x: auto;` Oculta el contenido que desborda.	Contenedores con imágenes grandes o para prevenir barras de desplazamiento.	Cuando el usuario necesita ver contenido truncado o scrollable.

⚠️ Incompatibilidad Crítica (Conflicto Directo)

`display: flex;` o `display: grid;` + `float: left / right / none;`

Cuando un elemento es declarado como contenedor Flex o Grid (`display: flex` o `grid`), **la propiedad `float` pierde completamente su efecto** en sus hijos directos. Aplicar `float` en elementos hijos dentro de un contenedor flexible es un error sintáctico lógico común que debe evitarse.

2. Tipografía, Visibilidad y Transformaciones

Comando / Propiedad	Sintaxis y Uso	Cuándo Sí Usar	Cuándo NO Usar
`visibility: hidden;`	`visibility: hidden;` Oculta el elemento pero mantiene su espacio físico.	Cuando se requiere ocultar un componente temporalmente reservando su espacio en pantalla.	Si necesitas que el espacio del elemento sea ocupado por los elementos adyacentes.
`display: none;`	`display: none;` Elimina por completo el elemento del DOM visual.	Ocultar menús desplegables cerrados, elementos en diseño responsive móvil.	Si requieres que el elemento siga interactuando con el layout o animaciones de entrada.
`pointer-events: none;`	`pointer-events: none;` Desactiva la interacción del ratón/touch.	Capas decorativas superiores (overlays) que no deben bloquear clics inferiores.	En botones, enlaces o elementos interactivos que el usuario deba presionar.
`z-index: [numero];`	`z-index: 999;` Controla el orden de apilamiento en el eje Z.	Modales, menús de navegación fijos (headers sticky), tooltips.	Sin definir previamente `position: relative`, `absolute` o `fixed` (no funciona en estáticos).
`transform: scale();`	`transform: scale(1.1);` Modifica escala sin alterar el flujo del DOM.	Efectos visuales suaves al pasar el cursor (hover) y animaciones de alto rendimiento.	Para ajustar el tamaño real y definitivo de cajas o contenedores estáticos de texto.

> ⚠️ **Incompatibilidad Crítica: `z-index` y `position: static`**
> La propiedad `z-index` **NO SURTE EFECTO** en elementos cuyo `position` sea `static` (el valor por defecto). Para que un comando `z-index` funcione correctamente, el elemento **obligatoriamente** debe tener un posicionamiento explícito como `relative`, `absolute`, `fixed` o `sticky`.

> 💡 **Regla de Oro sobre Unidades y Herencia**
> Evita mezclar unidades fijas extremas con relativas de manera inconsistente en textos responsivos. Prefiere `rem` para tamaños de tipografía y espaciados generales con el fin de asegurar una correcta accesibilidad y escalabilidad en todos los navegadores modernos.